Log-Guided Root-Cause Code Localization from Production Issues

Anonymous Author(s)

***Abstract*—Modern software systems generate large volumes of logs during operation, and these logs are essential for diagnosing failures and resolving reported issues. In industrial practice, engineers often begin troubleshooting from logs, yet existing techniques depend on execution information, such as reproducible tests or traces, which is unavailable when an issue is first reported. Given an issue report, system logs and source-code snapshots, this challenge calls for techniques that identify the root cause and produce a ranked list of candidate defective code regions.**

***Index Terms*—Log analysis, fault localization, root cause analysis.**

I. Context and Background

Industrial software systems generate massive volumes of logs during operation, and these logs are often the first evidence available when users report failures or unexpected behaviors. In practice, engineers may only receive a natural language issue report and a large log bundle, without reproducible execution or debugger state. They must first manually inspect logs to identify the affected module and the defective code region, a process which is costly and error-prone. For example, a single user-visible failure may leave only retries and timeout messages. This forces engineers to infer which log events are causal and which are merely downstream effects before inspecting the relevant code.

This problem is especially difficult in large software systems, where logs differ across components in format, terminology and granularity. Moreover, the meaning of a log message is often implicit in the surrounding source code and logging intent. Existing source-level fault localization techniques [1] rely on failure-context and execution-derived information such as failing tests or coverage data, while log-based root cause analysis methods in cloud AIOps [2] do not directly target ranked localization of defective source-code regions from error-log messages. This challenge calls for practical techniques that connect log semantics with code semantics, scoping and localizing defective code directly from user-reported issues and runtime logs.

II. Key Technical Challenges

This section outlines the key technical challenges faced when conducting log-based root cause localization in industrial practice.

Evidence-to-code semantic retrieval. The solution must align user-reported behavior, runtime log events, logging statements and surrounding source code across call chains. In large codebases, modules may share similar log templates or exception messages, so the challenge is to preserve recall of the true defective region while reducing the search space to a compact ranked list.

Noisy and incomplete log evidence. Industrial log bundles often contain repeated messages, retries, unrelated background activity and downstream errors produced after the original fault. At the same time, the decisive log event may be missing or expressed only through a generic timeout message. A solution must therefore select failure-relevant evidence from noisy logs and remain robust when the available evidence only partially describes the faulty execution.

Root cause localization beyond symptoms. Error logs often show where a failure was detected rather than where it originated. Effective localization therefore requires distinguishing symptom-producing code from root-cause code by reasoning over execution context, component dependencies and possible control-flow or data-flow relations. This is particularly challenging for issues where the system does not crash but behaves contrary to user expectations.

III. Expected Technical Requirements

A solution should support an end-to-end workflow from user issue reports and runtime logs to root-cause code localization. The input consists of a natural-language issue description, logs and source code snapshot. The output should include: (i) the affected module, (ii) a ranked list of suspicious code regions and (iii) an explanation linking issue symptoms, selected log events and candidate code regions.

Log-to-code retrieval. The solution should provide a retrieval model or retrieval-augmented method that jointly understands and retrieves log semantics and code semantics.

Root cause localization. The solution should localize the code region where the root cause occurred, rather than the logging statement that reported the symptom.

Benchmark and evaluation. The solution should be evaluated on a benchmark containing issue descriptions, logs, source code and groundtruth faulty code regions, improving retrieval and localization accuracy over the strongest available baseline, using metrics such as Recall@K, MRR and Acc@K.

References

[1] M. N. Rafi, D. J. Kim, T.-H. Chen, and S. Wang, "A multi-agent approach to fault localization via graph-based retrieval and reflexion," *arXiv preprint arXiv:2409.13642*, 2024.

[2] Z. Wang, Z. Liu, Y. Zhang, A. Zhong, J. Wang, F. Yin, L. Fan, L. Wu, and Q. Wen, "Rcagent: Cloud root cause analysis by autonomous agents with tool-augmented large language models," in *Proceedings of the 33rd ACM international conference on information and knowledge management*, 2024, pp. 4966–4974.